

Laboratory Record

submitted as part of

Programming for Data Science Lab (MACSE502)

LAB ASSESEMENT-5

by



Reg No: 26MML0031
Name: Hasnain H. Makada

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

AUGUST 2026

2026 FALL SEMESTER - Programming for Data Science Lab (MACSE502) Record - INDEX

(Weeks 1 to 5)

Week No	Date	Sl. No.	Type	Topic	Title with download Link	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform operations: select Name and Age, filter Age>25, add Country, sort by Age descending, calculate average age.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records: ModelID, Accuracy, Algorithm. Perform: Select ModelID and Accuracy, Filter Accuracy>90%, Add Status Production Ready, Sort by Accuracy, Calculate average accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create DataFrame with missing values in Name, Age, City. Fill Age mean, drop rows where Name/City missing, convert Age to int, normalize Age min-max scaling.	14
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	Streaming platform movie records: MovieID, Rating, Genre. Create dataset with missing values, Fill Rating average, Remove rows missing MovieID/Genre, Convert Rating int, Normalize rating.	19
3	23-07-2026	5	PP	NumPy	Creating and Manipulating Arrays Using NumPy	Create 2D array shape (3,4) random 0-10. Calculate mean and std, slice first two rows last two columns, reshape to (4,3), element-wise multiplication.	30
3	23-07-2026	6	EP	NumPy	Warehouse Shipment and Logistic Analytics Using Numpy	Logistics company tracks shipments 4 warehouses x 7 weeks. Create 4x7 matrix. Compute average, std, slice first two warehouses last three weeks, reshape 7x4, element-wise multiplication with cost matrix.	35
4	30/07/2026	7	PP	Pandas, Numpy, Matplotlib	Feature Engineering and Exploratory Data Analysis	Create DataFrame Feature1, Feature2, Target random data. Create log Feature1, bin Feature2 low/medium/high, correlation matrix, scatter plot Feature1 vs Feature2 colored by Target.	46
4	30/07/2026	8	EP	Pandas, NumPy, Matplotlib	Employee Attrition Prediction Using Workforce Indicators	Company wants analyze workforce: OvertimeHours, JobSatisfactionScore, SalaryLevel, TrainingHours, WorkLifeBalance, YearsAtCompany, PromotionCount, AttritionStatus. Create Log_OvertimeHours, Categorize JobSatisfaction, correlation matrix, Treemap.	56
5	06/08/2026	9	PP	Web Scraping requests, BeautifulSoup pandas json	Scrape data from webpage and store in structured format like CSV or JSON	Scrape data from a webpage and store it in structured format like CSV or JSON. URL: https://quotes.toscrape.com	63
5	06/08/2026	10	EP	Selenium Pandas Matplotlib	Online Job Market Analytics	A recruitment agency wants to analyze job opportunities posted on dynamic job portal. Develop Python app using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary, store in DataFrame, save CSV, visualize job demand by location. Portal: https://www.naukri.com	70

PP: Practice Program; EP: Exercise Program; CP: Challenging Program

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Title	Web Scraping and Data Collection			
Problem Statement	Scrape data from a webpage and store it in a structured format like CSV or JSON format			
Objectives	• To understand the concept of web scraping and data collection. • To retrieve webpage content using the Requests library. • To extract required information from HTML using BeautifulSoup. • To organize the scraped data using Pandas DataFrame. • To store the collected data in structured formats such as CSV and JSON.			
Dataset URL Link / File	https://quotes.toscrape.com			
Function Details	Sl. No.	Function	Package	Description of Function
	1	requests.get()	requests	Retrieves the HTML content of a webpage
	2	BeautifulSoup()	bs4	Parses the HTML content of the webpage
	3	find_all()	BeautifulSoup	Finds all matching HTML elements
	4	find()	BeautifulSoup	Finds the first matching HTML element
	5	DataFrame()	pandas	Converts collected data into tabular format
	6	to_csv()	pandas	Stores data in CSV format
	7	json.dump()	json	Stores Python data in JSON format
	8	json.load()	json	Reads data from a JSON file
Steps/ Process/ Procedure/	1. Import the required libraries: requests, BeautifulSoup, pandas and json. 2. Specify the URL of the webpage to be scraped.			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Algorithm/ Flowchart/ Model	3. Send an HTTP GET request using requests.get(). 4. Parse the HTML content using BeautifulSoup. 5. Locate the required HTML elements containing quotes, authors and tags. 6. Extract the required data from the webpage. 7. Store the extracted data in a Python list. 8. Convert the collected data into a Pandas DataFrame. 9. Save the structured data in CSV format using to_csv(). 10. Save the same data in JSON format using json.dump(). 11. Display the collected data to verify the output.
Program	<pre> import requests from bs4 import BeautifulSoup import pandas as pd import json url = "https://quotes.toscrape.com/" response = requests.get(url) print("Status Code:", response.status_code) soup = BeautifulSoup(response.text, "html.parser") print("Website Title:", soup.title.text) quotes = [] for item in soup.find_all("div", class_="quote"): quote = item.find("span", class_="text").text author = item.find("small", class_="author").text tags = [tag.text for tag in item.find_all("a", class_="tag")] quotes.append({ "Quote": quote, "Author": author, "Tags": " ".join(tags) }) print("Number of quotes collected:", len(quotes)) </pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT).

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
df = pd.DataFrame(quotes)

df

df.to_csv("quotes.csv", index=False)

print("Data saved successfully in CSV format.")

with open("quotes.json", "w", encoding="utf-8") as file:
    json.dump(quotes, file, indent=4, ensure_ascii=False)

print("Data saved successfully in JSON format.")

csv_data = pd.read_csv("quotes.csv")

csv_data

all_quotes = []

page = 1

while True:

    url = f"https://quotes.toscrape.com/page/{page}/"

    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")

    items = soup.find_all("div", class_="quote")

    if not items:
        break

    for item in items:

        quote = item.find("span", class_="text").text
        author = item.find("small", class_="author").text

        tags = [
            tag.text
            for tag in item.find_all("a", class_="tag")
        ]

        all_quotes.append({
            "Quote": quote,
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT).

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

	<pre> "Author": author, "Tags": ", ".join(tags) }) page += 1 print("Total quotes collected:", len(all_quotes)) df = pd.DataFrame(all_quotes) df df.to_csv("quotes_dataset.csv", index=False) with open("quotes_dataset.json", "w", encoding="utf-8") as file: json.dump(all_quotes, file, indent=4, ensure_ascii=False) print("CSV and JSON files created successfully.") </pre>
Output Snapshots With Explanation/ Observations	The required libraries requests, BeautifulSoup, pandas and json are imported for web scraping, HTML parsing, data organization and structured data storage. <pre> 1 import requests 2 from bs4 import BeautifulSoup 3 import pandas as pd 4 import json </pre> <pre> 1 url = "https://quotes.toscrape.com/" 2 3 response = requests.get(url) 4 5 print("Status Code:", response.status_code) </pre> Status Code: 200 <i>Fig. 1.1: Importing Required Python Libraries</i>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT).

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

The webpage <https://quotes.toscrape.com/> is accessed using `requests.get()`. The status code 200 confirms that the webpage was successfully retrieved.

```
[7]: 1 quotes = []
      2
      3 for item in soup.find_all("div", class_="quote"):
      4
      5     quote = item.find("span", class_="text").text
      6     author = item.find("small", class_="author").text
      7
      8     tags = [
      9         tag.text
     10         for tag in item.find_all("a", class_="tag")
     11     ]
     12
     13 quotes.append({
     14     "Quote": quote,
     15     "Author": author,
     16     "Tags": ", ".join(tags)
     17 })
     18
     19 print("Number of quotes collected:", len(quotes))

Number of quotes collected: 10
```

Fig. 1.2: Accessing the Webpage

The HTML content of the webpage is parsed using BeautifulSoup. The webpage title confirms that the correct page has been successfully loaded.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT).

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
number of quotes collected: 10

[8]: 1 df = pd.DataFrame(quotes)
      2
      3 df

[8]:
```

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig. 1.3: Parsing the HTML and displaying the dataset Content

```
1 df.to_csv("quotes.csv", index=False)
2
3 print("Data saved successfully in CSV format.")

Data saved successfully in CSV format.
```

Fig. 1.4: Saving Data in CSV Format

The scraped data is successfully stored in CSV format using the pandas `to_csv()` function.

The scraped data is successfully stored in JSON format using the `json.dump()` function.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT).

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
1 with open("quotes.json", "w", encoding="utf-8") as file:
2     json.dump(quotes, file, indent=4, ensure_ascii=False)
3
4 print("Data saved successfully in JSON format.")
```

Data saved successfully in JSON format.

Fig. 1.5: Saving Data in JSON Format

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig 1.6: Displaying the saved json and csv

The scraped data is successfully stored in JSON format using the `json.dump()` function.

The saved CSV file is read using `pandas.read_csv()` and displayed to verify that the data has been stored correctly.

The saved JSON file is read using `json.load()` and the stored records are displayed successfully, confirming correct JSON storage.

Conclusions

The experiment was successfully completed by scraping data from the given webpage using Python. The webpage content was retrieved using the requests library and parsed using BeautifulSoup. The required information such as quotes, authors, and tags was extracted and organized using a Pandas DataFrame. Finally, the collected data was successfully stored in structured CSV and JSON formats for further analysis and use.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018
2. Python Program (Google Colab link /Github link)
3. Class lecture link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M. Professor, School of CSE, Vellore Institute of Technology (VIT)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

Title	Online Job Market Analytics			
Problem Statement	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.			
Objectives	• To understand web scraping from a dynamic job portal using Selenium. • To extract job title, company name, location and salary information from job listings. • To organize and clean the collected data using a Pandas DataFrame. • To save the structured job dataset in CSV format. • To perform location-wise analysis and visualize job distribution using Matplotlib.			
Dataset URL Link / File	Job Portal: https://www.naukri.com Search role used: Data Scientist Cities: Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi			
Functions Details	Sl. No.	Function	Package	Description of function
	1	Firefox()	selenium	Starts a Firefox browser session controlled by Selenium.
	2	driver.get()	selenium	Opens the specified job-portal URL in the automated browser.
	3	execute_script()	selenium	Executes JavaScript to scroll the dynamically loaded webpage.
	4	find_elements()	selenium	Finds all matching job-card elements on the page.
	5	find_element()	selenium	Finds a required field inside an individual job card.
	6	time.sleep()	time	Pauses execution briefly to allow dynamic content to load.
	7	DataFrame()	pandas	Converts the collected job records into tabular form.
	8	drop_duplicates()	pandas	Removes duplicate job records from the DataFrame.
	9	reset_index()	pandas	Resets row indexing after data cleaning.
	10	to_csv()	pandas	Saves the structured job dataset as a CSV file.
	11	value_counts()	pandas	Counts job records for each location for analysis.
	12	bar()/savefig()	matplotlib	Creates and saves the location-wise job-demand bar chart.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Selenium, Pandas, Matplotlib and supporting Python modules. 2. Define the job role, location and maximum number of job listings to collect. 3. Launch the browser using Selenium WebDriver and construct the Naukri search URL. 4. Open the dynamic job-search page and allow sufficient time for the page to load. 5. Scroll the webpage so that dynamically loaded job cards become available. 6. Locate job-card elements using CSS selectors. 7. Extract Job Title, Company Name, Location and Salary from each job card. 8. Store the extracted records in a Python list and convert them into a Pandas DataFrame. 9. Remove duplicate records and save the cleaned dataset as a CSV file. 10. Repeat the search for Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi. 11. Use value_counts() to prepare location-wise job analysis and create a bar chart. 12. Save the visualization and multi-city CSV file, display the final summary and close the browser. START -> Open Job Portal -> Scrape Listings -> DataFrame -> CSV -> Location Analysis -> Bar Chart -> STOP			
Program	<pre>import time import pandas as pd import matplotlib.pyplot as plt from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.firefox.options import Options from selenium.common.exceptions import NoSuchElementException JOB_ROLE = "data scientist" LOCATION = "bangalore"</pre>			

Student Reg. No: 26MML0031

Name: Hasnain Makada

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
MAX_JOBS = 30

options = Options()
driver = webdriver.Firefox(options=options)
driver.set_window_size(1400, 900)

def make_slug(text):
    return "-".join(text.lower().strip().split())

def get_text(card, selectors, default="Not Available"):
    for selector in selectors:
        try:
            element = card.find_element(By.CSS_SELECTOR, selector)
            text = element.text.strip()
            if text:
                return text
        except NoSuchElementException:
            pass
    return default

job_slug = make_slug(JOB_ROLE)
location_slug = make_slug(LOCATION)
url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"
print("Search URL:", url)

driver.get(url)
time.sleep(5)
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)

for i in range(3):
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2)
driver.execute_script("window.scrollTo(0, 0);")

cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-tuple")
print("Number of job cards found:", len(cards))

jobs = []
for card in cards[:MAX_JOBS]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    location = get_text(card, ["span.locwdth", ".locwdth", "[class*='location']"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
                       default="Not Disclosed")

    jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": location,
        "Salary": salary
    })

print("Jobs scraped:", len(jobs))
df = pd.DataFrame(jobs)
df = df.drop_duplicates().reset_index(drop=True)
print("Total unique jobs:", len(df))
print("Dataset Shape:", df.shape)
print(df.head(10))
print("Salary Information:")
print(df["Salary"].value_counts())

df.to_csv("job_market_data.csv", index=False)
print("CSV file saved successfully.")

csv_data = pd.read_csv("job_market_data.csv")
print(csv_data.head())

cities = ["bangalore", "hyderabad", "pune", "chennai", "mumbai", "delhi"]
all_jobs = []

for city in cities:
    city_slug = make_slug(city)
    url = f"https://www.naukri.com/{job_slug}-jobs-in-{city_slug}"
    print("Scraping:", city)
    driver.get(url)
```

Student Reg. No: 26MML0031

Name: Hasnain Makada

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
time.sleep(4)

city_cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-tuple")
print("Jobs found:", len(city_cards))

for card in city_cards[:10]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
                        default="Not Disclosed")
    all_jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": city.title(),
        "Salary": salary
    })

city_df = pd.DataFrame(all_jobs)
city_df = city_df.drop_duplicates().reset_index(drop=True)
print("Total jobs collected:", len(city_df))

location_analysis = city_df["Location"].value_counts().reset_index()
location_analysis.columns = ["Location", "Number of Jobs"]
print(location_analysis)

plt.figure(figsize=(10, 5))
plt.bar(location_analysis["Location"], location_analysis["Number of Jobs"])
plt.title("Data Scientist Job Demand by Location")
plt.xlabel("Location")
plt.ylabel("Number of Job Listings")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("job_demand_by_location.png", dpi=300, bbox_inches="tight")
plt.show()

city_df.to_csv("job_market_multi_city.csv", index=False)
print("Multi-city dataset saved successfully.")

print("ONLINE JOB MARKET ANALYTICS")
print("Job Role:", JOB_ROLE)
print("Total Jobs Analyzed:", len(city_df))
print("Cities Analyzed:", len(location_analysis))
print(location_analysis.to_string(index=False))

driver.quit()
print("Browser closed successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

```
Firefox started successfully.

[5]: def make_slug(text):
      return "-".join(text.lower().strip().split())

      job_slug = make_slug(JOB_ROLE)
      location_slug = make_slug(LOCATION)

      url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"
      print("Search URL:", url)

Search URL: https://www.naukri.com/data-scientist-jobs-in-bangalore

[6]: driver.get(url)

      time.sleep(5)

      print("Page Title:", driver.title)
      print("Current URL:", driver.current_url)

Page Title: Data Scientist Jobs In Bangalore - 7018 Data Scientist Job Vacancies
In Bangalore - Naukri.com
Current URL: https://www.naukri.com/data-scientist-jobs-in-bangalore

[7]: print("Website Title:")
      print(driver.title)
```

Fig. 1.1: Search URL and Job Portal Validation

The Selenium session generated the Naukri search URL for Data Scientist jobs in Bangalore and

Student Reg. No: 26MML0031

Name: Hasnain Makada

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

successfully retrieved the page title and current URL, confirming that the dynamic job portal was opened correctly.

```
)
time.sleep(2)

driver.execute_script(
    "window.scrollTo(0, 0);"
)

print("Page scrolling completed.")

Page scrolling completed.

[9]: cards = driver.find_elements(
    By.CSS_SELECTOR,
    "div.cust-job-tuple"
)

print("Number of job cards found:", len(cards))

Number of job cards found: 20

[10]: print(cards[0].text)

Data Scientist
Aditya Birla Management Corporation (ABMCPL)
4.1
2366 Reviews
7-10 Yrs
Bengaluru
Position Title - Data Scientist Job Location - BengaluruJob Purpose The role
focuses o...
Machine LearningDeep LearningPythonPredictive ModelingPysparkArtificial
IntelligenceNatural Language ProcessingNeural Networks
```

Fig. 1.2: Detection of Dynamic Job Cards

After scrolling the page, Selenium detected 20 job-card elements. The first job card displays details such as job title, company, experience, location and associated skills.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
jobs.append({
    "Job Title": title,
    "Company Name": company,
    "Location": location,
    "Salary": salary
})

print("Jobs scraped:", len(jobs))

Jobs scraped: 20

[14]: jobs[:3]

[14]: [{'Job Title': 'Data Scientist',
      'Company Name': 'Aditya Birla Management Corporation (ABMCPL)',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist/Software Developer',
      'Company Name': 'Xylem Water Solutions',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist III',
      'Company Name': 'Walmart',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'}]

[15]: df = pd.DataFrame(jobs)

df

[15]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist
8	Data Scientist GenAI Agentic AI LLM RAG
9	Data Scientist AIML
10	Data Scientist/Python/MLOps- 4th Sep(Fri)-6+ y...
11	Data scientist III
12	Data Scientist
13	Data Scientist/ Data Analyst
14	Walk-in Data Scientist

Fig. 1.3: Scraped Job Records and Pandas DataFrame

The scraper collected 20 listings and extracted the required Job Title, Company Name, Location and Salary fields. The records were then converted into a structured Pandas DataFrame.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
7           Pune, Bengaluru Not Disclosed
8 Hybrid - Pune, Bengaluru, Delhi / NCR Not Disclosed
9           Hyderabad, Chennai, Bengaluru Not Disclosed
```

```
[18]: print("Dataset Shape:", df.shape)

      print("\nColumn Names:")
      print(df.columns.tolist())

      print("\nTotal Records:", len(df))
```

Dataset Shape: (20, 4)

Column Names:
['Job Title', 'Company Name', 'Location', 'Salary']

Total Records: 20

```
[19]: print("Salary Information:")

      print(df["Salary"].value_counts())
```

Salary Information:
Salary
Not Disclosed 16
20-30 Lacs PA 1
15-20 Lacs PA 1
14-20 Lacs PA 1
0-18 Lacs PA 1
Name: count, dtype: int64

```
[20]: df.to_csv(
      "job_market_data.csv",
```

Fig. 1.4: Dataset Structure, Salary Availability and CSV Creation

The initial dataset contains 20 records and 4 required columns. Salary information was unavailable for most listings, while a few postings contained salary ranges. The cleaned DataFrame was saved successfully as job_market_data.csv.

```
      index=False
    )
    print("CSV file saved successfully.")
```

CSV file saved successfully.

```
[21]: csv_data = pd.read_csv(
      "job_market_data.csv"
    )

    csv_data.head()
```

```
[21]:      Job Title \
0           Data Scientist
1  Data Scientist/Software Developer
2           Data Scientist III
3           Data Scientist
4 Data Scientist | Python | GenAI | LLM | RAG

      Company Name \
0 Aditya Birla Management Corporation (ABMCPL)
1           Xylem Water Solutions
2           Walmart
3           Genpact
4           Infosys

      Location      Salary
0      Bengaluru Not Disclosed
1      Bengaluru Not Disclosed
2      Bengaluru Not Disclosed
3      Bengaluru 20-30 Lacs PA
```

Fig. 1.5: Verification of the Saved CSV Dataset

The generated CSV file was read back using pandas.read_csv(), and the displayed rows confirm that the job-listing data was stored correctly in structured form.

Student Reg. No: 26MML0031

Name: Hasnain Makada

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
"Company Name": company,  
"Location": city.title(),  
"Salary": salary  
})
```

```
Scraping: bangalore  
Jobs found: 20
```

```
Scraping: hyderabad  
Jobs found: 20
```

```
Scraping: pune  
Jobs found: 20
```

```
Scraping: chennai  
Jobs found: 20
```

```
Scraping: mumbai  
Jobs found: 21
```

```
Scraping: delhi  
Jobs found: 21
```

```
[25]: city_df = pd.DataFrame(all_jobs)  
  
city_df = city_df.drop_duplicates().reset_index(drop=True)  
  
print("Total jobs collected:", len(city_df))  
  
city_df.head(10)
```

Total jobs collected: 60

```
[25]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist

Fig. 1.6: Multi-City Job Data Collection

The same Data Scientist search was executed for Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi. A controlled sample of 10 listings per city produced 60 records for location-wise analysis.

```
4 Infosys Bangalore Not Disclosed  
5 Hsbc Bangalore Not Disclosed  
6 ExxonMobil Bangalore Not Disclosed  
7 VOIS Bangalore Not Disclosed  
8 Infosys Bangalore Not Disclosed  
9 Tata Consultancy Services Bangalore Not Disclosed
```

```
[26]: location_analysis = (  
    city_df["Location"]  
    .value_counts()  
    .reset_index()  
)  
  
location_analysis.columns = [  
    "Location",  
    "Number of Jobs"  
)  
  
location_analysis
```

```
[26]:
```

	Location	Number of Jobs
0	Bangalore	10
1	Hyderabad	10
2	Pune	10
3	Chennai	10
4	Mumbai	10
5	Delhi	10

```
[27]: plt.figure(figsize=(10, 5))
```

Fig. 1.7: Location-Wise Job Analysis

Student Reg. No: 26MML0031

Name: Hasnain Makada

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

Pandas value_counts() produced the location-wise summary for the six-city job sample.

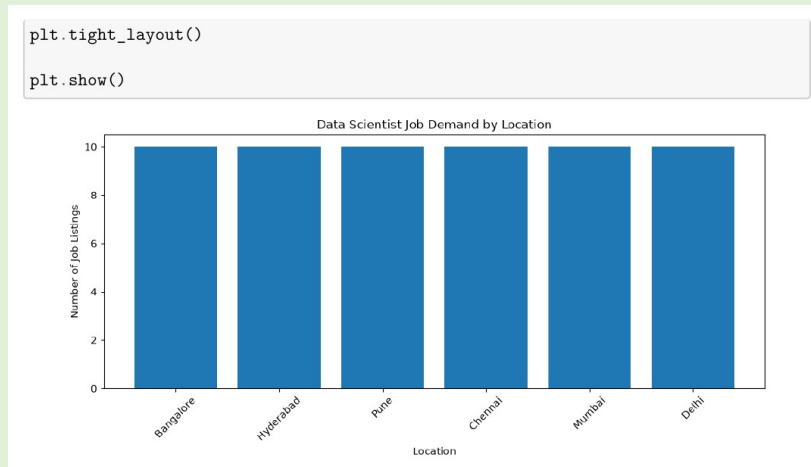


Fig. 1.8: Data Scientist Job Demand Visualization by Location

A Matplotlib bar chart visualizes the number of sampled job listings for each selected city. The graph provides a clear location-wise comparison of the collected job-market sample.

```
[30]: print("ONLINE JOB MARKET ANALYTICS")
      print("-" * 35)

      print("Job Role:", JOB_ROLE)
      print("Total Jobs Analyzed:", len(city_df))
      print("Cities Analyzed:", len(location_analysis))

      print("\nLocation-wise Job Demand:")
      print(location_analysis.to_string(index=False))

      ONLINE JOB MARKET ANALYTICS
      -----
      Job Role: data scientist
      Total Jobs Analyzed: 60
      Cities Analyzed: 6

      Location-wise Job Demand:
      Location Number of Jobs
      Bangalore          10

      Hyderabad          10
      Pune               10
      Chennai            10
      Mumbai             10
      Delhi              10

[31]: driver.quit()
      print("Browser closed successfully ")
```

Fig. 1.9: Final Job Market Analytics Summary

The final output confirms that the multi-city dataset and graph were saved successfully. The analysis includes 60 sampled job records across 6 cities and displays the location-wise summary.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

Conclusions	The exercise successfully demonstrated Selenium-based web scraping on a dynamic job portal. Job title, company name, location and available salary information were extracted from Naukri, organized in Pandas DataFrames, cleaned and stored as CSV files. Location-wise analysis was performed for six cities and visualized using a Matplotlib bar chart. Thus, the required job listings dataset, CSV output, location analysis and demand visualization were generated successfully.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018. 2. Python Program (GitHub link): _____ 3. Class lecture link: https://drive.google.com/file/d/1HjCacfaFs49XEnuM3idduuKB0Tgjb4xI/view

Student Reg. No: 26MML0031

Name: Hasnain Makada